

New National Advertising

Production CMS + Public Website Sync & Favicon Fix

IMPORTANT: This is a production CMS/data-flow FIX for New National Advertising.

The Admin Panel login and Git synchronization are currently working.

DO NOT BREAK THE EXISTING AUTHENTICATION, ADMIN AUTHORIZATION, Firebase configuration, or any other currently working feature.

The main problem now is:

WHEN I EDIT, ADD, REMOVE, PUBLISH, UNPUBLISH, OR CHANGE CONTENT FROM THE ADMIN PANEL, THE CHANGE IS NOT RELIABLY APPEARING ON THE ORIGINAL PUBLIC WEBSITE.

FIX THE ACTUAL DATA FLOW. DO NOT JUST CHANGE THE ADMIN UI.

1. ADMIN PANEL MUST CONTROL THE LIVE WEBSITE

The Admin Panel must be the real source of CMS-managed website content.

Required flow:

Admin Panel

- authenticated API request
- backend validation
- Firestore/database update
- successful response
- public API reads updated record
- public website displays updated record

Every successful Admin change must persist after:

- Admin page refresh
- browser refresh
- opening the public website in a new tab
- opening the public website in an incognito/private window

Do not use local React state as the permanent source of truth.

Do not save only to localStorage.

Do not show "Saved successfully" unless the backend/database actually saved the change.

2. AUDIT THE ENTIRE ADMIN → PUBLIC DATA FLOW

Inspect the existing code before changing it.

For every Admin CMS module, identify:

- Admin form/component
- API request
- backend route
- backend controller/service
- Firestore collection/document
- response returned by backend
- public API route
- public frontend component
- caching/query layer
- any hardcoded fallback/static data

Find exactly where the connection is breaking.

Fix the root cause rather than adding another duplicate data system.

3. EVERYTHING CMS-MANAGED MUST BE EDITABLE

PRODUCTS

- name
- image
- description
- category
- price if supported
- status/stock if supported
- publish/unpublish
- ordering
- archive/remove

MACHINES

- name
- image
- description
- specifications
- ordering
- publish/unpublish
- archive/remove

SERVICES

- service name
- service image
- description

- slug
- ordering
- publish/unpublish
- archive/remove

SERVICE PRODUCTS / MATERIALS

These must be individually editable.

Examples:

- Star Flex
- Star Black Back
- One Way Vision
- Canvas
- Gloss Vinyl
- Matt Vinyl
- Vinyl with Sunboard
- Vinyl with Sunpack
- Sunboard 3mm / 5mm
- Backlight Printing

For each service product/material, Admin must be able to:

- change name
- change image
- upload/replace image
- edit description where supported
- change ordering
- publish/unpublish
- add new item
- archive/remove

PROJECTS

- title
- description
- images
- service relationship
- featured status
- publish/unpublish
- ordering
- archive/remove

WEBSITE CONTENT

Where currently implemented:

- hero heading
- hero text
- hero image
- CTA text/links
- About content
- contact information
- phone
- WhatsApp
- email
- address
- social links
- footer content
- homepage sections
- other CMS-managed content

4. SERVICE PRODUCTS MUST ACTUALLY APPEAR ON PUBLIC SERVICE PAGES

If I edit:

Star Flex

→ Premium Star Flex

and save it in Admin,

the public service page MUST show:

Premium Star Flex

after refresh.

If I replace the image, the public page must show the new image.

If I add a new service product, it must appear publicly when published.

If I unpublish a service product, it must disappear from public content.

There must be no hardcoded React array overriding database data.

5. REMOVE CONFLICTING HARDCODED DATA

Search the whole frontend for hardcoded CMS content.

Check for:

- static product arrays

- static service arrays
- static service-product arrays
- static machine arrays
- static project arrays
- hardcoded contact details
- duplicate homepage data
- mock/demo data
- fallback data that overrides valid database data

Replace conflicting hardcoded CMS data with API/database data.

Legitimate static UI configuration can remain static.

Do NOT remove static design/layout configuration unnecessarily.

6. PUBLIC WEBSITE MUST FETCH CURRENT DATA

Audit all public pages:

- Home
- Services
- Service detail pages
- Products
- Product detail pages if present
- Machines
- Projects / Our Work
- Contact
- About
- other CMS-managed pages

Verify that every CMS-managed value comes from the current backend data.

The public website must not use stale build-time data when the content is supposed to be dynamic.

7. CACHE / REFRESH PROBLEM

Check whether stale data is caused by:

- React Query cache
- browser cache
- API caching
- localStorage
- static build data
- memoized values
- stale component state

- server-side caching
- incorrect fetch timing

After a successful Admin mutation:

- invalidate the affected cache
- refetch the updated data
- update the Admin UI
- ensure the public API returns the new data

Do NOT introduce unnecessary realtime complexity if normal API refetch/invalidation is enough.

8. IMAGE UPLOADS MUST PERSIST

When an Admin changes an image:

1. Upload image to persistent Firebase Storage/existing storage
2. Get persistent URL/reference
3. Save that URL/reference in Firestore
4. Return updated record
5. Public API returns the new image
6. Public website displays the new image

Do NOT use temporary blob URLs as permanent image URLs.

If upload fails:

- keep the old image
- show an error
- do not report success

9. CRUD MUST BE REAL

CREATE:

Admin creates record

- backend creates record
- database confirms
- Admin refreshes
- record remains
- public website sees it if published

UPDATE:

Admin edits record

- backend updates database
- response returns updated record

- Admin refreshes
- updated record remains
- public website shows updated record

UNPUBLISH:

Admin unpublishes

- database status changes
- public API excludes it
- public website no longer displays it

ARCHIVE/REMOVE:

Admin archives/removes

- database confirms
- Admin list updates
- public content updates accordingly

10. ERROR HANDLING

For every save/update/delete operation:

Before request:

- show loading state
- disable duplicate submission

On success:

- show success
- refresh/revalidate data

On failure:

- show clear error
- keep form values
- do not show fake success
- do not corrupt existing data

Handle:

- 400
- 401
- 403
- 404
- 500
- Firebase errors
- Storage errors

- network errors

11. FAVICON FIX

The current favicon technically exists, but it is too small/unclear in the browser tab.

Create a dedicated SIMPLE favicon.

Do NOT use the full horizontal New National Advertising logo with tiny text.

Use a simplified recognizable New National Advertising / NN brand mark.

Requirements:

- large central mark
- square composition
- high contrast
- minimal details
- no tiny text
- recognizable at 16x16 and 32x32

Create/maintain:

- favicon.svg
- favicon-16x16.png
- favicon-32x32.png
- favicon.ico
- apple-touch-icon.png where appropriate

Update the production HTML correctly.

Use correct favicon references in the HTML head, such as:

```
<link rel="icon" type="image/svg+xml" href="/favicon.svg">
<link rel="icon" type="image/png" sizes="32x32" href="/favicon-32x32.png">
<link rel="icon" type="image/png" sizes="16x16" href="/favicon-16x16.png">
<link rel="shortcut icon" href="/favicon.ico">
```

Make sure these assets actually exist in the production build.

Verify the production URLs return the actual favicon files and not 404/HTML.

Do NOT change the existing header logo.

12. DO NOT BREAK AUTHENTICATION

ABSOLUTELY DO NOT MODIFY OR REPLACE:

- Firebase authentication

- Firebase ID token verification
- Admin authorization
- admin role checking
- /admin/login
- /admin session handling
- Render Firebase environment configuration
- existing working security middleware

The admin login is already working.

Leave it alone.

13. DO NOT CREATE DUPLICATE DATABASE SYSTEMS

Reuse the existing:

- Firestore collections
- Firebase Storage
- API routes
- backend services
- frontend API client
- TypeScript types

Do not create a second collection just because an existing one is inconvenient.

Do not create duplicate API routes for the same resource unless necessary.

14. END-TO-END TESTS ARE REQUIRED

TEST A: SERVICE

Admin changes service name

- Save
- Refresh Admin
- Open public website
- Verify new name

TEST B: SERVICE IMAGE

Admin changes service image

- Save
- Refresh
- Verify new image publicly

TEST C: SERVICE PRODUCT

Admin changes:

Star Flex

→ Premium Star Flex

Save

→ refresh

→ public service page

Verify:

Premium Star Flex

TEST D: SERVICE PRODUCT IMAGE

Replace image

→ save

→ public page

→ verify new image

TEST E: ADD PRODUCT

Add new service product

→ publish

→ public service page

→ verify it appears

TEST F: UNPUBLISH

Unpublish service product

→ public page

→ verify it disappears

TEST G: MAIN PRODUCT

Edit product name/image

→ save

→ public Products page

→ verify changes

TEST H: MACHINE

Edit machine

→ save

→ public Machines/homepage

→ verify changes

TEST I: PROJECT

Create/edit/publish project

→ public Projects/homepage

→ verify changes

TEST J: WEBSITE CONTENT

Edit one homepage/content field

→ save

→ public website

→ verify change

TEST K: PERSISTENCE

Make an edit

→ refresh browser

→ verify edit remains

TEST L: NEW BROWSER

Make an edit

→ open public website in a new/incognito browser

→ verify updated value

TEST M: IMAGE PERSISTENCE

Upload image

→ refresh

→ verify image remains

TEST N: FAILED REQUEST

Cause/handle failed save

→ verify old data remains safe

→ verify no fake success

TEST O: FAVICON

Production website

→ hard refresh

→ verify the simplified favicon is clearly visible in the browser tab

15. BUILD + PRODUCTION VERIFICATION

Run:

- TypeScript typecheck
- production build
- lint if configured
- verify API routes
- verify Firebase/Firestore

- verify Firebase Storage
- verify production asset paths

Do not report the issue as fixed based only on local preview.

Verify the actual deployed production website.

16. FINAL REPORT

At the end, provide:

1. Root cause of why Admin changes were not appearing on the public website
2. Files/components changed
3. API routes fixed
4. Firestore collections used
5. Public pages connected
6. Service-product relationship
7. Image storage flow
8. Cache/revalidation fix
9. Favicon fix
10. End-to-end tests performed
11. Any remaining issue

MOST IMPORTANT:

The Admin Panel is the control panel for the LIVE WEBSITE.

If I change something in Admin and save it successfully, the public website MUST use the changed data.

Do not fake the result by changing only the Admin UI.

Fix the actual backend/database/public-frontend data flow.

Do not claim completion until the end-to-end tests have passed.